AJAY.I - 71812401007 · AGENTIC AI COGNITION · MODULE 1 — ADVANCED REASONING FOR AI AGENTS NOTES

1. REASONING PARADIGMS

1.1 What is Reasoning in AI?

Simple explanation:

Reasoning is nothing but the ability of an AI system to think about a problem, understand the available information and decide what to do.

For example, imagine you ask an AI agent:

"Find the largest number among 10, 25, 15 and 40."

The agent must:

1. Understand the question.
2. Examine the given numbers.
3. Compare the numbers.
4. Identify the largest number.
5. Provide the answer.

This process of analyzing information and arriving at a conclusion is called reasoning.

Technical explanation:

Reasoning in AI refers to the computational process through which an intelligent system analyzes information, identifies relationships, evaluates alternatives and derives conclusions to solve a problem or achieve a goal.

In Agentic AI, reasoning helps an agent decide which actions to perform, which tools to use and how to respond to changing situations.

1.2 Types of Reasoning Paradigms

The important reasoning approaches include:

1. Deductive Reasoning

Simple explanation:

Deductive reasoning means starting with a general rule and applying it to a particular situation.

Example:

- All humans need oxygen.
- Ajay is a human.
- Therefore, Ajay needs oxygen.

Technical explanation:

Deductive reasoning derives a specific conclusion from general rules or established premises. If the premises are true and the reasoning is logically valid, the conclusion must be true.

2. Inductive Reasoning

Simple explanation:

Inductive reasoning means observing multiple examples and identifying a general pattern.

Example:

- The sun rose yesterday.
- The sun rose today.
- The sun has risen every day you observed.
- Therefore, the sun is expected to rise tomorrow.

Technical explanation:

Inductive reasoning identifies general patterns or rules from specific observations. Its conclusions are probable rather than logically guaranteed.

3. Abductive Reasoning

Simple explanation:

Abductive reasoning means finding the most likely explanation for something that has happened.

Example:

- Your mobile phone is not switching on.
- Its battery may be empty.
- Therefore, a discharged battery is one possible explanation.

Technical explanation:

Abductive reasoning involves inferring the most plausible explanation for an observed event or set of facts.

4. Analogical Reasoning

Simple explanation:

Analogical reasoning means solving a new problem by comparing it with a similar problem you already understand.

Example:

If you know how to operate one smartphone, you can understand many features of another smartphone because they have similar interfaces.

Technical explanation:

Analogical reasoning transfers knowledge or solutions from a familiar situation to a new situation based on similarities between them.

1.3 Comparison of Reasoning Types

Type	Meaning	Example
Deductive	General rule to specific conclusion	All birds have feathers; a parrot is a bird.
Inductive	Specific observations to general pattern	Repeated observations suggest a pattern.
Abductive	Find the most likely explanation	Wet road suggests possible rainfall.
Analogical	Use similarities to solve a problem	Learn a new application using a familiar interface.

Importance in Agentic AI:

An AI agent may use deductive reasoning to follow rules, inductive reasoning to identify patterns, abductive reasoning to diagnose problems and analogical reasoning to reuse previous solutions.

2. CHAIN OF THOUGHT (CoT)

2.1 What is Chain of Thought?

Simple explanation:

Chain of Thought is nothing but solving a problem step by step instead of directly jumping to the final answer.

Imagine someone asks:

"A shopkeeper buys a product for ₹500 and sells it for ₹650. Find the profit."

Instead of directly answering ₹150, the AI breaks the problem into smaller steps:

1. Cost price = ₹500.
2. Selling price = ₹650.
3. Profit = Selling price − Cost price.
4. Profit = 650 − 500 = ₹150.

This step-by-step approach is called Chain of Thought.

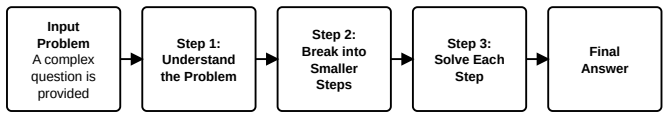
Technical explanation:

Chain of Thought (CoT) is a prompting and reasoning technique that encourages a language model to generate intermediate reasoning steps before producing its final answer.

It helps the model decompose complex problems into smaller logical steps.

2.2 Working of Chain of Thought

DIAGRAM



Working of Chain of Thought

2.3 Example

Question:

"A student scores 80 marks in Mathematics and 70 marks in Physics. Find the average."

Using Chain of Thought:

- Step 1: Identify the marks.
- Step 2: Add the marks: 80 + 70 = 150.
- Step 3: Count the subjects: 2.
- Step 4: Average = 150 / 2.
- Final answer = 75.

2.4 Types of Chain of Thought

1. Zero-Shot Chain of Thought

Simple explanation: Ask the AI to solve the problem step by step without giving it an example.

Example prompt:

```
Solve this problem step by step:
If 5 pens cost ₹100, what is the
cost of 8 pens?
```

The model generates intermediate reasoning steps.

2. Few-Shot Chain of Thought

Simple explanation: Give the AI a few examples of step-by-step solutions and then ask it to solve a new problem in the same way.

Example:

```
Example 1:
2 + 3 = 5

Example 2:
4 + 6 = 10

Now solve:
8 + 7 = ?
```

The examples guide the model's response format and reasoning approach.

2.5 Advantages

- Breaks complex problems into smaller steps.
- Helps solve mathematical and logical problems.
- Improves structured problem-solving.
- Makes intermediate work easier to inspect when provided.
- Supports multi-step reasoning.

2.6 Limitations

- May generate incorrect intermediate steps.
- Can increase response time and token usage.
- Long reasoning may introduce unnecessary information.
- Step-by-step output does not guarantee correctness.

Important: Chain of Thought is a reasoning technique, not a guarantee that every generated reasoning step is correct.

3. PROGRAM OF THOUGHT (PoT)

3.1 What is Program of Thought?

Simple explanation:

Program of Thought is nothing but allowing an AI model to write a small program to solve a problem instead of performing every calculation mentally.

For example, if you ask an AI to calculate the average of 1,000 numbers, it can generate Python code to calculate the result.

Instead of manually calculating everything, the model creates a program and lets the computer execute it.

Technical explanation:

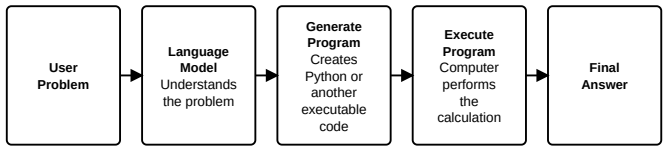
Program of Thought (PoT) is a reasoning technique in which a language model translates a problem into executable code.

The generated program is executed using an external interpreter or execution environment, and the result is used to produce the final answer.

It separates reasoning and problem formulation from numerical or computational execution.

3.2 Working of PoT

DIAGRAM



Working of Program of Thought

3.3 Python Example

Question:

"Find the average of 10, 20, 30, 40 and 50."

The AI can generate this program:

```
numbers = [10, 20, 30, 40, 50]
total = sum(numbers)
average = total / len(numbers)
print("Average:", average)
```

Output:

```
Average: 30.0
```

Explanation

- `numbers` stores the given values.
- `sum()` calculates their total.
- `len()` counts the values.
- Division calculates the average.
- `print()` displays the result.

3.4 Applications

- Mathematical calculations.
- Statistical analysis.
- Data processing.
- Scientific computations.
- Financial calculations.
- Automated problem-solving.

3.5 Advantages

- Accurate execution of supported calculations.
- Useful for complex mathematical problems.
- Can process large datasets.
- Reuses existing programming libraries.
- Reduces errors caused by manual calculations.

3.6 Limitations

- Generated code may contain errors.
- Requires a safe execution environment.
- Incorrect program logic produces incorrect results.
- External execution may increase processing time.

4. SKELETON OF THOUGHT (SoT)

4.1 What is Skeleton of Thought?

Simple explanation:

Skeleton of Thought is nothing but first creating the basic structure of an answer and then filling in the details.

Imagine you are writing a 10-mark answer.

Instead of writing everything immediately, you first prepare the headings:

1. Introduction.
2. Architecture.
3. Working.
4. Advantages.
5. Applications.

Then you explain each heading.

This is the basic idea of Skeleton of Thought.

Technical explanation:

Skeleton of Thought (SoT) is a prompting technique in which a language model first generates a high-level outline or structural skeleton of a response and subsequently expands each section with relevant details.

It separates the organization of information from detailed content generation.

4.2 Working of SoT

DIAGRAM



Working of Skeleton of Thought

4.3 Example

Question:

"Explain the applications of Agentic AI."

First, the model creates a skeleton:

```
1. Introduction
2. Healthcare
3. Education
4. Transportation
5. Manufacturing
6. Finance
7. Conclusion
```

Next, it expands each heading with relevant explanations and examples.

4.4 Advantages

- Organizes information clearly.

- Helps generate structured answers.
- Reduces the chance of missing major sections.
- Useful for reports and long-form content.
- Separates planning from detailed writing.

4.5 Limitations

- The initial outline may be incomplete.
- Detailed expansion may introduce irrelevant information.
- It does not independently guarantee factual correctness.

5. COMPARISON: CoT vs PoT vs SoT

This is an important comparison for examinations.

Feature	Chain of Thought	Program of Thought	Skeleton of Thought
Full form	CoT	PoT	SoT
Main idea	Solve step by step	Generate executable code	Generate an outline first
Focus	Logical reasoning	Computational execution	Structured response
Uses code?	Not necessarily	Yes, generally	Not necessary
Best suited for	Logical problems	Mathematical and computational problems	Long structured answers
Example	Calculate profit step by step	Use Python to calculate profit	Create headings before writing

Easy way to remember

- CoT: Think step by step.
- PoT: Write a program and execute it.
- SoT: Prepare the structure and expand it.

6. SELF-CORRECTION AND REFLECTION FRAMEWORK

6.1 What is Self-Correction?

Simple explanation:

Self-correction is nothing but an AI agent checking its own answer, identifying mistakes and correcting them.

For example, an AI coding agent writes a Python program.

When it executes the program, it receives an error.

It examines the error, changes the code and runs the program again.

Technical explanation:

Self-correction is a mechanism in which an AI agent evaluates its generated output or actions, detects errors using feedback or verification and generates revised outputs to improve task performance.

6.2 What is Reflection?

Simple explanation:

Reflection means looking back at previous work and thinking about what went wrong and how to improve it.

For example, after completing a report, an AI agent checks whether all required sections are present and improves the report if something is missing.

Technical explanation:

Reflection is an agent reasoning mechanism that analyzes previous actions, decisions and outcomes to identify weaknesses and generate improvements for future behavior.

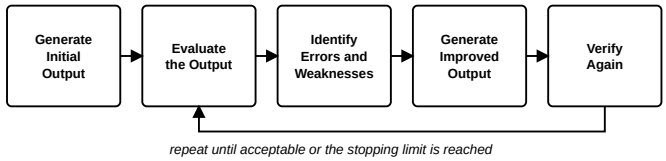
6.3 Self-Correction vs Reflection

Self-Correction	Reflection
Focuses on correcting errors.	Focuses on evaluating previous work.
Produces a revised output.	Identifies lessons and weaknesses.
Often uses verification and feedback.	Uses critique and analysis.
Example: Fixing a coding error.	**Example:** Evaluating why the code failed.

Reflection can be used as part of a self-correction process.

6.4 Reflection Framework Working

DIAGRAM



Reflection framework working

The process can repeat until the output satisfies the required conditions or the stopping limit is reached.

6.5 Types of Self-Correction

1. Feedback-Based Correction

Simple explanation: The agent uses feedback to identify mistakes.

Example: A robot receives a signal that its movement was unsuccessful and changes its next action.

2. Execution-Based Correction

Simple explanation: The agent runs its output and checks whether it works.

Example: A coding agent executes Python code and corrects runtime errors.

3. Reflection-Based Correction

Simple explanation: The agent analyzes its previous answer and identifies weaknesses.

Example: A report-writing agent notices missing information and adds it.

4. Verification-Based Correction

Simple explanation: The agent compares its result with expected conditions.

Example: An agent checks whether a generated JSON response contains all required fields.

6.6 Python Example

```python
answer = 10 * 6
expected = 60

if answer == expected:
    print("Correct answer")
else:
    print("Error detected")
    answer = expected

print("Final answer:", answer)
```

Output:

```
Correct answer
Final answer: 60
```

This is a basic verification example. Real AI self-correction also needs an evaluation mechanism and a way to revise the generated solution.

6.7 Advantages

- Improves output quality.
- Helps identify errors.
- Supports continuous improvement.
- Useful in coding and planning agents.
- Reduces repeated mistakes when lessons are retained.

Limitation: An AI agent can incorrectly evaluate its own answer. Independent testing and external verification are important.

7. ACTOR–EVALUATOR ARCHITECTURE

7.1 What is Actor–Evaluator Architecture?

Simple explanation:

Actor–Evaluator Architecture is nothing but dividing an AI system into two parts:

- Actor: Performs the task.
- Evaluator: Checks whether the task was performed correctly.

Imagine a student writing an examination.

The student writes the answer. A teacher evaluates it and points out the mistakes.

The actor–evaluator architecture follows a similar approach.

Technical explanation:

Actor–Evaluator Architecture is a design pattern in which one component generates actions or solutions, while another component evaluates their quality against predefined objectives, constraints or feedback criteria.

The evaluator's feedback can be used by the actor to generate an improved solution.

7.2 Main Components

1. Actor

Simple explanation: The actor performs the task.

Technical explanation: The actor is responsible for generating responses, selecting actions, executing plans or producing solutions based on the available input.

Example: An AI coding agent generates a Python program.

2. Evaluator

Simple explanation: The evaluator checks the actor's work.

Technical explanation: The evaluator examines the generated solution using evaluation criteria such as correctness, quality, safety and task requirements.

Example: Another component executes the code and checks its test results.

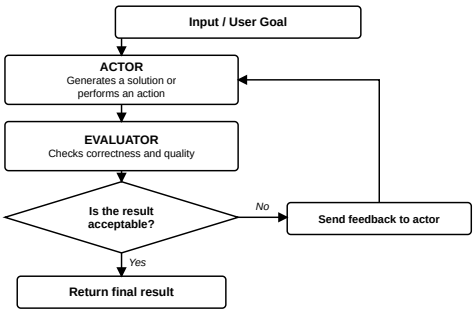
3. Feedback Mechanism

Simple explanation: The evaluator tells the actor what needs to be improved.

Technical explanation: The feedback mechanism communicates evaluation results to the actor to support revision and optimization.

7.3 Architecture Diagram

DIAGRAM



Actor–Evaluator architecture

If the output is rejected, the actor revises its solution and submits it for evaluation again.

7.4 Example

Suppose an AI agent must develop a calculator application.

Actor:

1. Generates the Python code.
2. Implements addition, subtraction and multiplication.
3. Produces the initial program.

Evaluator:

1. Runs the program.
2. Checks whether addition works.
3. Checks whether subtraction works.
4. Identifies any errors.
5. Sends feedback to the actor.

Actor again:

Corrects the errors and generates an improved program.

7.5 Advantages

- Separates generation and evaluation.
- Improves solution quality.
- Supports iterative correction.
- Helps detect errors.
- Useful for software development and autonomous agents.

7.6 Applications

- Code generation and testing.
- Automated report generation.
- Mathematical problem-solving.
- AI planning.
- Content verification.

Important: Actor generates; evaluator judges; feedback helps the actor improve.

8. PLANNING STRATEGIES

8.1 What is Planning in AI?

Simple explanation:

Planning is nothing but deciding what steps an AI agent must follow to achieve a particular goal.

For example, suppose you want an AI agent to book a train ticket.

It cannot complete everything in one step.

It must:

1. Understand the destination.
2. Search available trains.
3. Compare timings.
4. Select a suitable train.
5. Enter passenger information.
6. Obtain confirmation before making a payment if required.
7. Complete the permitted booking operation.

This is planning.

Technical explanation:

Planning in AI is the process of identifying a sequence of actions that transforms an initial state into a desired goal state while considering available actions, constraints and environmental conditions.

8.2 Components of Planning

Component	Meaning
Initial State	The current condition of the environment.
Goal State	The desired final condition.
Actions	Operations available to the agent.
Preconditions	Conditions required before an action.
Effects	Changes produced by an action.
Plan	Ordered sequence of actions.

Example

A robot must move from Room A to Room C.

- Initial state: Room A.
- Goal state: Room C.
- Available actions: Move right, move left.
- Plan: Room A → Room B → Room C.

8.3 Types of Planning Strategies

1. Forward Planning

Simple explanation:

Forward planning starts from the current situation and decides which steps will lead to the goal.

Example:

A robot is in Room A and wants to reach Room D.

It determines:

Room A → Room B → Room C → Room D.

Technical explanation:

Forward planning, also called progression planning, begins from the initial state and explores possible actions and resulting states until a goal state is reached.

2. Backward Planning

Simple explanation:

Backward planning starts from the final goal and works backward to identify the steps required to achieve it.

Example:

Goal: Submit a completed assignment.

Work backward:

- Assignment must be submitted.
- Assignment must be completed.
- Questions must be answered.
- Study material must be collected.

Technical explanation:

Backward planning, also called regression planning, begins with the goal conditions and identifies actions and preceding conditions required to achieve them.

3. Hierarchical Planning

Simple explanation:

Hierarchical planning means breaking a large task into smaller tasks.

Example:

Goal: Organize a college event.

Break it into:

- Select venue.
- Arrange food.
- Invite guests.
- Prepare schedule.
- Conduct the event.

Technical explanation:

Hierarchical planning decomposes a high-level task into smaller subtasks and primitive actions using a hierarchy of goals and operations.

4. Reactive Planning

Simple explanation:

Reactive planning means changing the plan immediately when something unexpected happens.

Example:

A self-driving car changes its route when it detects an obstacle.

Technical explanation:

Reactive planning selects or modifies actions based on current observations and environmental changes rather than relying entirely on a fixed sequence of actions.

5. Goal-Based Planning

Simple explanation:

The agent selects actions based on the final goal it needs to achieve.

Example:

A navigation agent selects a route to reach the destination.

Technical explanation:

Goal-based planning searches for an action sequence that satisfies a specified goal condition.

8.4 Planning Strategy Comparison

Strategy	Starting point	Main idea	Example
Forward	Current state	Move toward goal	Robot moves to destination
Backward	Goal state	Identify required previous steps	Plan backward from submission
Hierarchical	Large goal	Divide into subtasks	Organize an event
Reactive	Current observation	Change actions dynamically	Avoid an obstacle
Goal-based	Desired goal	Select actions to achieve goal	Book a ticket

9. REACT REASONING PATTERN

9.1 What is ReAct?

Simple explanation:

ReAct is nothing but an AI agent thinking about what to do, performing an action, observing the result and thinking again.

For example, an AI agent needs to find the latest weather information.

It:

1. Thinks about what information is required.
2. Calls a weather tool.
3. Receives the result.
4. Examines the result.
5. Provides the answer.

Technical explanation:

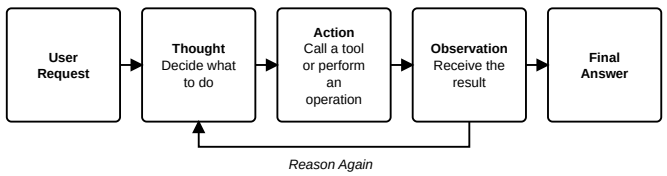
ReAct stands for Reasoning and Acting.

It is an agent reasoning framework that combines language-model reasoning with external actions and environmental observations.

The agent alternates between reasoning and acting to solve multi-step problems.

9.2 ReAct Architecture

DIAGRAM



ReAct architecture

9.3 Example

User asks:

"Find the current price of a product online."

Thought: I need current product information.

Action: Search the product website.

Observation: The website returns product details and price.

Thought: The price information is available.

Final answer: Provide the product name and retrieved price.

9.4 Advantages

- Combines reasoning and action.
- Supports external tool usage.
- Handles multi-step tasks.
- Adapts based on observations.
- Useful for autonomous agents.

9.5 Applications

- Web searching.
- Software development.
- Customer support.
- Data analysis.
- Browser automation.
- Robotics.

10. TREE OF THOUGHTS (ToT)

10.1 What is Tree of Thoughts?

Simple explanation:

Tree of Thoughts is nothing but an AI agent exploring multiple possible solutions instead of following only one line of thinking.

Imagine you want to travel from Coimbatore to Chennai.

There are multiple possible routes.

Instead of immediately selecting one route, the agent explores different routes, compares them and chooses the most suitable one.

This is the basic idea of Tree of Thoughts.

Technical explanation:

Tree of Thoughts (ToT) is a reasoning framework in which a language model explores multiple intermediate reasoning paths organized as a search tree.

The model generates candidate thoughts, evaluates their potential and explores promising branches to identify a suitable solution.

10.2 Why is ToT needed?

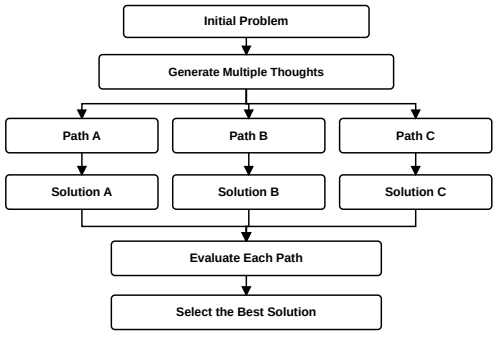
Chain of Thought generally follows one reasoning path.

But sometimes the first approach may not work.

Tree of Thoughts allows the agent to explore multiple alternatives before selecting a solution.

10.3 Working of Tree of Thoughts

DIAGRAM



Working of Tree of Thoughts

10.4 Example

Question:

"How can a delivery company reduce delivery time?"

The AI generates three possible approaches:

- Path A: Use the shortest geographical route.
- Path B: Use a route with less traffic.
- Path C: Divide deliveries among multiple vehicles.

The agent evaluates these approaches based on distance, traffic and available vehicles.

It then selects the most suitable solution.

10.5 Important Components

1. Thought Generation

The model generates multiple possible reasoning paths.

2. Evaluation

Each reasoning path is evaluated based on its usefulness or progress toward the goal.

3. Search

The system explores promising branches and may discard weak branches.

4. Final Selection

The best available solution is selected.

10.6 Advantages

- Explores multiple solutions.
- Useful for complex decision-making.
- Avoids depending entirely on one reasoning path.
- Supports comparison and evaluation.
- Helps solve problems with multiple possible approaches.

10.7 Limitations

- Requires more computation.
- Can increase response time.
- Evaluating multiple paths can be difficult.
- May explore unnecessary alternatives.

11. REASONING VIA PLANNING (RAP)

11.1 What is RAP?

Simple explanation:

RAP is nothing but solving a problem by treating reasoning as a planning task.

Instead of simply generating an answer, the AI agent considers:

- Where am I currently?
- What is my goal?
- What actions can I perform?
- Which sequence of actions will reach the goal?

For example, a robot must reach a destination.

It examines its current location, available movements and the destination before selecting the next action.

Technical explanation:

Reasoning via Planning (RAP) is a framework that formulates language-model reasoning as a planning problem.

It represents possible reasoning steps or actions as states and transitions, then searches for a sequence of actions that leads to a desired goal.

RAP uses planning concepts to guide language-model reasoning, rather than relying only on free-form text generation.

11.2 Working of RAP

Define the Problem → Identify Initial State and Goal → Generate Possible Actions → Search and Evaluate Plans → Select a Suitable Plan

11.3 Example

A robot must move from Room A to Room D.

Possible actions:

- Move left.
- Move right.
- Move forward.
- Move backward.

The agent identifies the initial state and goal state, explores available actions and determines a suitable path.

The planning process helps it select a sequence of movements.

11.4 Advantages

- Provides structured reasoning.
- Supports goal-oriented problem-solving.
- Explores alternative action sequences.
- Useful in complex environments.
- Combines language-model reasoning with planning.

12. LANGUAGE AGENT TREE SEARCH (LATS)

12.1 What is LATS?

Simple explanation:

LATS is nothing but an AI agent exploring multiple possible actions, checking their results and selecting the best path.

Imagine playing chess.

You cannot simply make the first move that comes to your mind.

You need to:

1. Consider possible moves.
2. Think about what happens after each move.
3. Evaluate the resulting positions.
4. Choose the best move.
5. Repeat the process.

LATS applies a similar approach to language-based agents.

Technical explanation:

Language Agent Tree Search (LATS) is a framework that combines language-model reasoning, planning, search, evaluation and environmental feedback.

It uses tree-search methods, commonly Monte Carlo Tree Search (MCTS), to explore possible action sequences and select promising decisions.

It also incorporates observations and self-evaluation to improve agent performance.

12.2 Main Components

1. Language Model

Generates possible thoughts and actions.

2. Search Tree

Stores different possible action paths.

3. Environment

Executes actions and provides observations.

4. Evaluation

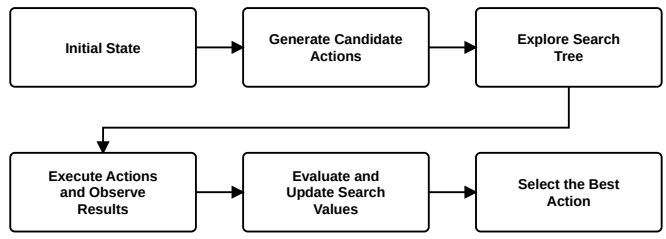
Evaluates the quality of generated actions and resulting states.

5. Backpropagation

Updates the values of explored search paths based on their results.

12.3 Working of LATS

DIAGRAM

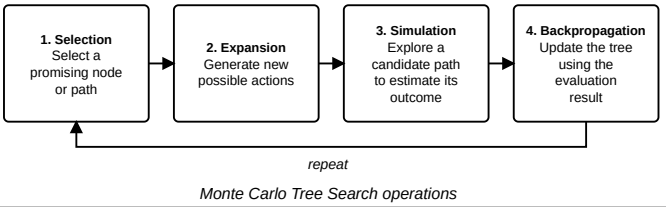


Working of LATS

12.4 Important Search Operations

LATS uses concepts from Monte Carlo Tree Search.

DIAGRAM



Monte Carlo Tree Search operations

1. Selection

Select a promising node or path in the search tree.

2. Expansion

Generate new possible actions or reasoning paths.

3. Simulation

Explore or execute a candidate path to estimate its outcome.

4. Backpropagation

Update the search tree using the evaluation result.

12.5 Example

Suppose an AI agent must solve a coding problem.

It:

1. Generates multiple possible solutions.
2. Explores different approaches.
3. Executes selected code.
4. Observes the output.
5. Evaluates correctness.
6. Updates the search values.
7. Selects the best-performing solution.

12.6 Advantages

- Explores alternative solutions.
- Uses feedback from the environment.
- Supports complex reasoning.
- Combines planning and evaluation.
- Can improve decision quality.

12.7 Limitations

- Requires substantial computation.
- Search may become expensive.
- Depends on the quality of evaluation.
- Incorrect feedback can lead to poor decisions.

13. COMPARISON OF ADVANCED REASONING TECHNIQUES

This is one of the most important comparison areas in Module 1.

Technique	Main concept	How it solves a problem
CoT	Step-by-step reasoning	Follows a reasoning sequence
PoT	Program-based reasoning	Generates and executes code
SoT	Structural reasoning	Creates an outline and expands it
ReAct	Reasoning and acting	Thinks, acts and observes
ToT	Multiple reasoning paths	Explores and evaluates alternatives
RAP	Planning-based reasoning	Searches for a sequence of actions
LATS	Tree-search-based reasoning	Explores actions using search and feedback

Important differences

CoT vs ToT

- CoT follows a reasoning path step by step.
- ToT explores multiple possible reasoning paths.

ToT vs LATS

- ToT explores and evaluates different thought paths.
- LATS combines tree search with action execution, environmental feedback and evaluation.

RAP vs ReAct

- ReAct alternates between reasoning and external actions.
- RAP formulates reasoning as a planning problem and searches for a sequence of actions.

PoT vs CoT

- CoT uses intermediate reasoning steps.
- PoT generates executable programs to perform computational tasks.

14. IMPORTANT 2-MARK QUESTIONS

Prepare these definitions for short-answer questions.

1. Define Reasoning in AI.

Reasoning in AI is the process of analyzing information, identifying relationships and deriving conclusions to solve problems or make decisions.

2. What is Chain of Thought (CoT)?

Chain of Thought is a reasoning technique in which a language model solves a complex problem through intermediate reasoning steps before producing a final answer.

3. What is Program of Thought (PoT)?

Program of Thought is a reasoning technique in which a language model generates executable code to solve computational problems.

4. What is Skeleton of Thought (SoT)?

Skeleton of Thought is a prompting technique in which a model first generates an outline and then expands it into a complete response.

5. What is Zero-Shot CoT?

Zero-Shot CoT is a technique in which a model is instructed to reason step by step without being provided with worked examples.

6. What is Few-Shot CoT?

Few-Shot CoT provides examples of step-by-step reasoning to guide a model in solving a new problem.

7. Define Self-Correction.

Self-correction is the process by which an AI agent evaluates its output, identifies errors and generates an improved result using feedback or verification.

8. What is Reflection in AI?

Reflection is the process of analyzing previous actions or outputs to identify weaknesses and improve future performance.

9. What is Actor–Evaluator Architecture?

Actor–Evaluator Architecture is a design pattern in which an actor generates solutions and an evaluator checks their quality and provides feedback for improvement.

10. Define Planning in AI.

Planning is the process of identifying a sequence of actions required to transform an initial state into a desired goal state.

11. What is Forward Planning?

Forward planning starts from the initial state and explores actions toward the goal state.

12. What is Backward Planning?

Backward planning starts from the desired goal and identifies the actions and conditions required to achieve it.

13. What is Hierarchical Planning?

Hierarchical planning divides a large task into smaller subtasks and lower-level actions.

14. Expand ReAct.

ReAct stands for Reasoning and Acting.

15. What is Tree of Thoughts?

Tree of Thoughts is a reasoning framework that explores multiple possible reasoning paths and evaluates them to identify a suitable solution.

16. Expand RAP.

RAP stands for Reasoning via Planning.

17. What is LATS?

LATS stands for Language Agent Tree Search. It combines language-model reasoning, tree search, evaluation and environmental feedback.

18. Expand MCTS.

MCTS stands for Monte Carlo Tree Search. It is a search algorithm that explores possible decision paths using repeated simulations and evaluations.